COMPARATIVO ARQUITETURAL: MONÓLITO, ARQUITETURA EM CAMADAS (MVC) E CLEAN ARCHITECTURE EM SISTEMAS ESCALÁVEIS

ARCHITECTURAL COMPARISON: MONOLITH, LAYERED ARCHITECTURE (MVC) AND CLEAN ARCHITECTURE IN SCALABLE SYSTEMS

RESUMO: Este trabalho tem como objetivo comparar, de forma quantitativa, o desempenho e a manutenibilidade de sistemas de software estruturados em três padrões arquiteturais: Monólito, Arquitetura em Camadas (MVC) e Clean Architecture. A metodologia consiste em um protótipo tripartido, no qual a mesma aplicação de gerenciamento de tarefas é desenvolvida três vezes, mantendo constantes o domínio, a linguagem (TypeScript com Node.js), o banco de dados (PostgreSQL) e a infraestrutura de execução (Docker, em um único hospedeiro), com carga gerada pelo autocannon e métricas coletadas da mesma forma em cada versão. Em quatro cenários, com 5 rodadas cada, não houve diferença detectável de desempenho na leitura e no cenário misto; na criação de tarefas, o módulo único alcançou 1.060 req/s contra 1.001 da Clean Architecture ($p = 0{,}005$). Na estrutura do código, os mesmos testes cobriram 78% da regra de negócio no módulo único, 96% em camadas e 94% na Clean Architecture, cuja camada de domínio não importa nenhum framework (contra 50% em camadas), enquanto o acoplamento medido por fan-out cresceu com o número de camadas. Os resultados se limitam a uma aplicação pequena em um único hospedeiro.
Palavras-chave: Arquitetura de Software; Desempenho; Escalabilidade; Manutenibilidade; Testabilidade.

ABSTRACT: This work aims to compare, quantitatively, the performance and maintainability of software systems structured in three architectural patterns: Monolith, Layered Architecture (MVC) and Clean Architecture. The methodology is a tripartite prototype, in which the same task management application is developed three times, keeping the domain, the language (TypeScript with Node.js), the database (PostgreSQL) and the execution infrastructure (Docker, on a single host) constant, with load generated by autocannon and metrics collected the same way in every version. In four scenarios, with 5 rounds each, no detectable performance difference was found for reads and for the mixed scenario; when creating tasks, the single module reached 1.060 req/s against 1.001 for Clean Architecture ($p = 0{,}005$). Regarding code structure, the same tests covered 78% of the business logic in the single module, 96% in layers and 94% in Clean Architecture, whose domain layer imports no framework (versus 50% in layers), while coupling measured by fan-out grew with the number of layers. The results are limited to a small application on a single host.
Keywords: Maintainability; Performance; Scalability; Software Architecture; Testability.

1 INTRODUÇÃO

A transformação digital ampliou a demanda sobre a arquitetura dos sistemas de software, que precisam sustentar grande volume de solicitações simultâneas e disponibilidade constante. A escolha do padrão arquitetural é uma decisão estratégica, pois afeta a evolução, a manutenção e a escalabilidade do sistema durante todo o seu ciclo de vida (Garlan; Shaw, 1993). Padrões como a Arquitetura em Camadas, difundida no modelo MVC (*Model-View-Controller*), e a Clean Architecture (Martin, 2017) favorecem a divisão de responsabilidades, a testabilidade e a independência de tecnologias externas.

Escolher o padrão que ofereça o melhor equilíbrio entre facilidade de implementação, manutenibilidade e escalabilidade em sistemas de médio porte é o problema central desta pesquisa. Essa escolha costuma se apoiar em preferências pessoais, modismos ou relatos anedóticos (Fowler, 2002), o que gera decisões subótimas, dívida técnica e custos crescentes de manutenção (Newman, 2015). Blinowski, Ojdowska e Przybylek (2022) mostraram que, em máquina única, o monólito supera os microsserviços em latência, o que reforça a necessidade de medição rigorosa. Nas fontes revisadas (Seção 3.2), não foram encontrados estudos que comparem os três padrões sob as mesmas condições operacionais.

O objetivo deste trabalho é analisar quantitativamente o desempenho e a manutenibilidade de sistemas construídos segundo os três padrões, por meio de um protótipo funcional tripartido executado em contêineres Docker, com métricas coletadas de forma idêntica em cada versão.

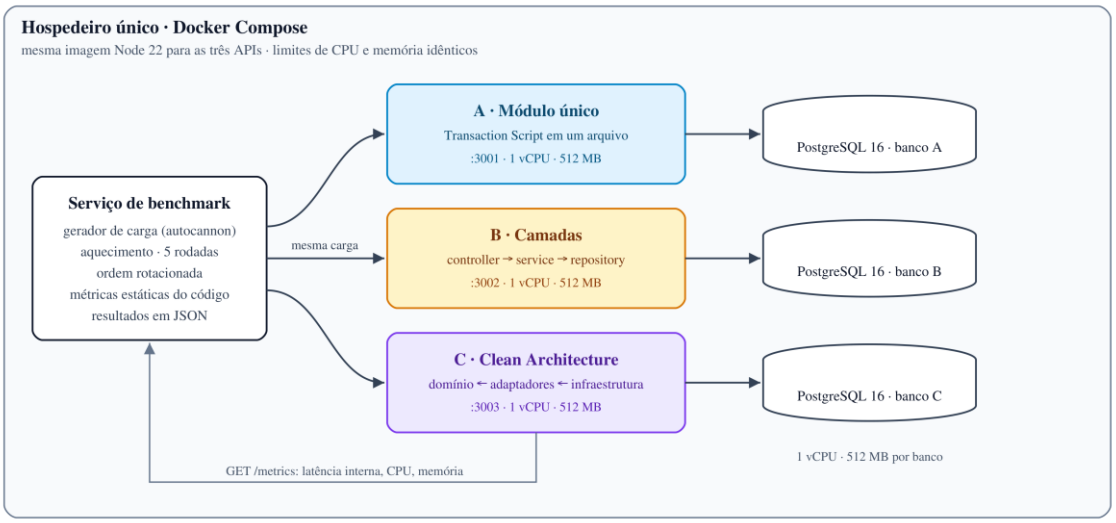
2 METODOLOGIA

A pesquisa adota a implementação de um protótipo tripartido: a mesma aplicação de gerenciamento de tarefas é desenvolvida três vezes, cada versão seguindo um padrão (A: Monólito, em módulo único; B: Arquitetura em Camadas/MVC; C: Clean Architecture). Assim, o padrão arquitetural é a variável independente, enquanto o domínio de negócio, a linguagem (TypeScript com Node.js), o banco de dados (PostgreSQL) e a infraestrutura de execução (Docker) permanecem constantes. Cada versão contempla API REST, regras de negócio para tarefas, autenticação por *JSON Web Tokens* (JWT) e persistência em PostgreSQL, diferindo apenas na organização do código-fonte e na distribuição de responsabilidades. A equivalência funcional é verificada por um teste que executa o mesmo roteiro HTTP nas três versões e exige respostas idênticas.

Cada versão é executada em um contêiner Docker próprio e tem seu banco PostgreSQL 16 em outro contêiner, todos com o mesmo limite de 1 vCPU e 512 MB, em um único hospedeiro (Windows 11, 8 processadores lógicos, Docker 29.4.3), conforme a Figura 1. O gerador de carga autocannon, no mesmo hospedeiro, executa quatro cenários (listar, buscar, criar e misto) com 10 conexões simultâneas, 3 s de aquecimento e 10 s de medição, em 5 rodadas com a ordem das versões rotacionada a cada rodada. Tempo de resposta e *throughput* são medidos pelo

gerador, e o consumo de CPU por requisição é lido de cada API. Indicadores de qualidade de código, como cobertura de testes, dependências entre módulos e linhas de código por camada, complementam a análise. A diferença entre as versões é testada com o teste de Kruskal-Wallis (Kruskal; Wallis, 1952), adequado a amostras de tamanho 5 sem suposição de normalidade, com nível de significância de 0,05.

Figura 1 – Ambiente experimental e instrumentação da coleta de métricas



Fonte: Elaborado pelos autores (2026).

3 REFERENCIAL TEÓRICO

3.1 PADRÕES ARQUITETURAIS

A arquitetura de software estabelece a organização de um sistema por meio de decisões estruturais e princípios de projeto. Garlan e Shaw (1993) a definem como o estudo da organização global dos componentes, de suas interações e dos princípios que governam o projeto e a evolução do sistema; para Bass, Clements e Kazman (2012), é o conjunto de estruturas necessárias para raciocinar sobre o sistema, seus elementos e as relações entre eles. O modelo monolítico, que reúne interface, lógica de negócio e persistência em um único artefato implantável, é historicamente o mais utilizado: proporciona baixa latência em contextos de baixa complexidade, mas limita o escalonamento seletivo e amplia o acoplamento à medida que o sistema cresce (Dragoni *et al.*, 2017).

A Arquitetura em Camadas, popularizada pelo MVC, reduz o acoplamento por meio da separação horizontal entre apresentação, lógica de negócio e persistência,

melhorando a organização do código e facilitando a escalabilidade (Tu, 2023). A Clean Architecture (Martin, 2017) isola as regras de negócio no núcleo de camadas concêntricas e aplica o princípio da inversão de dependências, o que favorece a testabilidade e a independência tecnológica.

3.2 TRABALHOS RELACIONADOS

A comparação empírica entre padrões arquiteturais é uma linha de investigação estabelecida, porém concentrada no eixo monólito–microsserviços. O trabalho metodologicamente mais próximo é o de Blinowski, Ojdowska e Przybylek (2022), que compararam, de forma controlada, uma aplicação monolítica e sua versão em microsserviços, mantendo constantes a funcionalidade e o ambiente de execução. Em máquina única, a versão monolítica apresentou menor latência, atribuída à comunicação em processo, enquanto a distribuída foi superior no escalonamento horizontal sob carga crescente. Esse achado fundamenta a adoção das mesmas métricas (tempo de resposta, *throughput* e consumo de recursos) neste trabalho. Tapia *et al.* (2020) seguiram desenho semelhante, com a mesma funcionalidade sob arquiteturas distintas, medida em condições equivalentes, com foco em consumo de recursos de hardware e latência, procedimento que este trabalho replica.

Revisões sistemáticas recentes reforçam essa leitura: Di Francesco, Lago e Malavolta (2019) mapearam 103 estudos primários sobre arquitetura de microsserviços e evidenciaram a predominância desse eixo; Abgaz *et al.* (2023) analisaram 35 estudos sobre decomposição de monólitos e concluíram que o tema permanece em estágio inicial de maturidade; e Velepucha e Flores (2023) sistematizaram princípios e desafios de migração. A literatura compara majoritariamente monólitos e microsserviços, sem considerar a Clean Architecture como alternativa estruturalmente coesa (Richards, 2015). Nenhum dos estudos identificados a avalia empiricamente em comparação controlada com o monólito e com a Arquitetura em Camadas, sob identidade funcional, de linguagem e de infraestrutura, lacuna em que se insere este trabalho.

4 RESULTADOS E DISCUSSÃO

A implementação das três versões foi concluída, e os mesmos oito casos de teste de unidade passam em todas. Os resultados dos cenários de carga estão na Tabela 1 e os indicadores de código na Tabela 2.

Tabela 1 – Cenários de carga (média ± desvio-padrão, n = 5; 10 conexões; 1 vCPU) e valor p de Kruskal-Wallis

Cenário	Métrica	A	B	C	p
Listar	req/s	1.090 ± 143	1.066 ± 100	1.067 ± 86	0,651
	latência (ms)	8,8 ± 1,5	8,9 ± 0,9	8,9 ± 0,8	0,651
Buscar	req/s	1.384 ± 241	1.383 ± 181	1.405 ± 168	0,932
	latência (ms)	6,9 ± 1,6	6,8 ± 1,0	6,7 ± 0,9	0,932
Criar	req/s	1.060 ± 11	1.030 ± 23	1.001 ± 29	0,005
	latência (ms)	8,9 ± 0,1	9,2 ± 0,2	9,5 ± 0,3	0,008
Misto	req/s	1.042 ± 204	1.076 ± 67	1.064 ± 31	0,595
	latência (ms)	9,0 ± 1,5	8,8 ± 0,6	8,9 ± 0,3	0,595

Fonte: Elaborado pelos autores (2026). A: módulo único; B: camadas; C: Clean Architecture.

Nos cenários de leitura (listar e buscar) e no misto, não houve diferença detectável entre as versões ($p \geq 0{,}59$), mas o coeficiente de variação do *throughput* chegou a 20%, o que indica ruído do hospedeiro e limita o poder do teste. A exceção foi a criação de tarefas, medida com baixa variação (CV ≤ 3%): o módulo único teve o maior *throughput* (1.060 req/s), seguido de Camadas (1.030) e Clean Architecture (1.001), diferença de 5,5% entre os extremos ($p = 0{,}005$), com o custo de CPU por requisição na mesma ordem. A hipótese de menor latência do monólito em baixa carga, derivada de Blinowski, Ojdowska e Przybylek (2022), confirmou-se, portanto, apenas na escrita e com efeito pequeno.

Tabela 2 – Indicadores de qualidade de código das três versões

Indicador	A	B	C
Arquivos de código	1	16	29
Linhas de código (LOC)	205	356	565
Fan-out médio (imports internos por arquivo)	0,00	1,63	2,76
Dependências entre camadas	0	23	43
Cobertura total de linhas	78%	42%	38%
Cobertura da camada de negócio	78%	96%	94%
Arquivos de negócio que importam frameworks	100%	50%	0%

Fonte: Elaborado pelos autores (2026). Cobertura dos mesmos oito casos de teste, sem banco de dados.

A Clean Architecture tem 2,8 vezes mais linhas que o módulo único (565 contra 205) e o maior fan-out médio (2,76 contra 0); a hipótese de menor acoplamento entre módulos, medido por esse indicador, não se confirmou. Já a independência de tecnologias externas, medida pela parcela de arquivos de negócio que importam frameworks ou drivers, foi de 100% no módulo único, 50% em

camadas e 0% na Clean Architecture. A testabilidade, medida pela cobertura da camada de negócio com os mesmos testes, foi de 78%, 96% e 94%, respectivamente.

5 CONSIDERAÇÕES FINAIS

Em relação ao objetivo proposto, as três versões foram implementadas, verificadas como funcionalmente equivalentes e comparadas. Nas condições medidas, o desempenho foi equivalente na leitura e até 5,5% menor na escrita das versões mais estruturadas, enquanto a camada de negócio de Camadas e da Clean Architecture teve cobertura maior e, na Clean Architecture, nenhuma dependência de frameworks; o acoplamento entre módulos, por outro lado, cresceu com o número de camadas. Os resultados têm validade limitada: o gerador de carga e as APIs dividem o mesmo hospedeiro, a aplicação é pequena, cada padrão foi implementado uma vez e há apenas 5 rodadas por condição.

Como trabalhos futuros, propõem-se avaliar a escalabilidade horizontal, repetir o estudo com outras implementações e aplicações maiores e incluir pipelines de integração e entrega contínua (CI/CD) para cada versão.

REFERÊNCIAS BIBLIOGRÁFICAS

ABGAZ, Y. *et al.* Decomposition of monolith applications into microservices architectures: a systematic review. **IEEE Transactions on Software Engineering**, v. 49, n. 8, p. 4213-4242, 2023. DOI: 10.1109/TSE.2023.3287297.

BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software architecture in practice**. 3. ed. Boston: Addison-Wesley, 2012.

BLINOWSKI, G.; OJDOWSKA, A.; PRZYBYŁEK, A. Monolithic vs. microservice architecture: a performance and scalability evaluation. **IEEE Access**, v. 10, p. 20357-20374, 2022. DOI: 10.1109/ACCESS.2022.3152803.

DI FRANCESCO, P.; LAGO, P.; MALAVOLTA, I. Architecting with microservices: a systematic mapping study. **Journal of Systems and Software**, v. 150, p. 77-97, 2019. DOI: 10.1016/j.jss.2019.01.001.

DRAGONI, N. *et al.* Microservices: yesterday, today, and tomorrow. *In*: MAZZARA, M.; MEYER, B. (ed.). **Present and ulterior software engineering**. Cham: Springer, 2017. p. 195-216.

FOWLER, M. **Patterns of enterprise application architecture**. Boston: Addison-Wesley, 2002.

GARLAN, D.; SHAW, M. An introduction to software architecture. *In*: AMBRIOLA, V.; TORTORA, G. (ed.). **Advances in software engineering and knowledge engineering**. Singapore: World Scientific Publishing, 1993. p. 1-39.

KRUSKAL, W. H.; WALLIS, W. A. Use of ranks in one-criterion variance analysis. **Journal of the American Statistical Association**, v. 47, n. 260, p. 583-621, 1952. DOI: 10.1080/01621459.1952.10483441.

MARTIN, R. C. **Clean architecture**: a craftsman's guide to software structure and design. Upper Saddle River: Prentice Hall, 2017.

NEWMAN, S. **Building microservices**: designing fine-grained systems. Sebastopol: O'Reilly Media, 2015.

RICHARDS, M. **Software architecture patterns**. Sebastopol: O'Reilly Media, 2015.

TAPIA, F. *et al.* From monolithic systems to microservices: a comparative study of performance. **Applied Sciences**, v. 10, n. 17, art. 5797, 2020. DOI: 10.3390/app10175797.

TU, Z. Research on the application of layered architecture in computer software development. **Journal of Computing and Electronic Information Management**, v. 11, n. 3, p. 34-38, 2023. DOI: 10.54097/jceim.v11i3.08.

VELEPUCHA, V.; FLORES, P. A survey on microservices architecture: principles, patterns and migration challenges. **IEEE Access**, v. 11, p. 88339-88358, 2023. DOI: 10.1109/ACCESS.2023.3305687.